CSE470 → quiz (SDLC)

CSE423 → " (DDA, MPL, Eight way algo) → March 4

CSE321 → " (Process and thread) → March 3

CSE470

Waterfall Model

Sequencial methodolgy for software project management.

Steps: Requirement analysis →
Design →
Coding →
Testing →
Development →
Maintenance

(changes only here + no feature add)

When to use → 1. Requirements are well-known.
2. Short scale and short-term project.
3. Resources are available and trained.
4. Technological tools used are static, not dynamic
5. When there's no scope of future expension.

Advantages → 1. Simple and easy
2. Changes in Development phase only, so no need to go back and change everything.

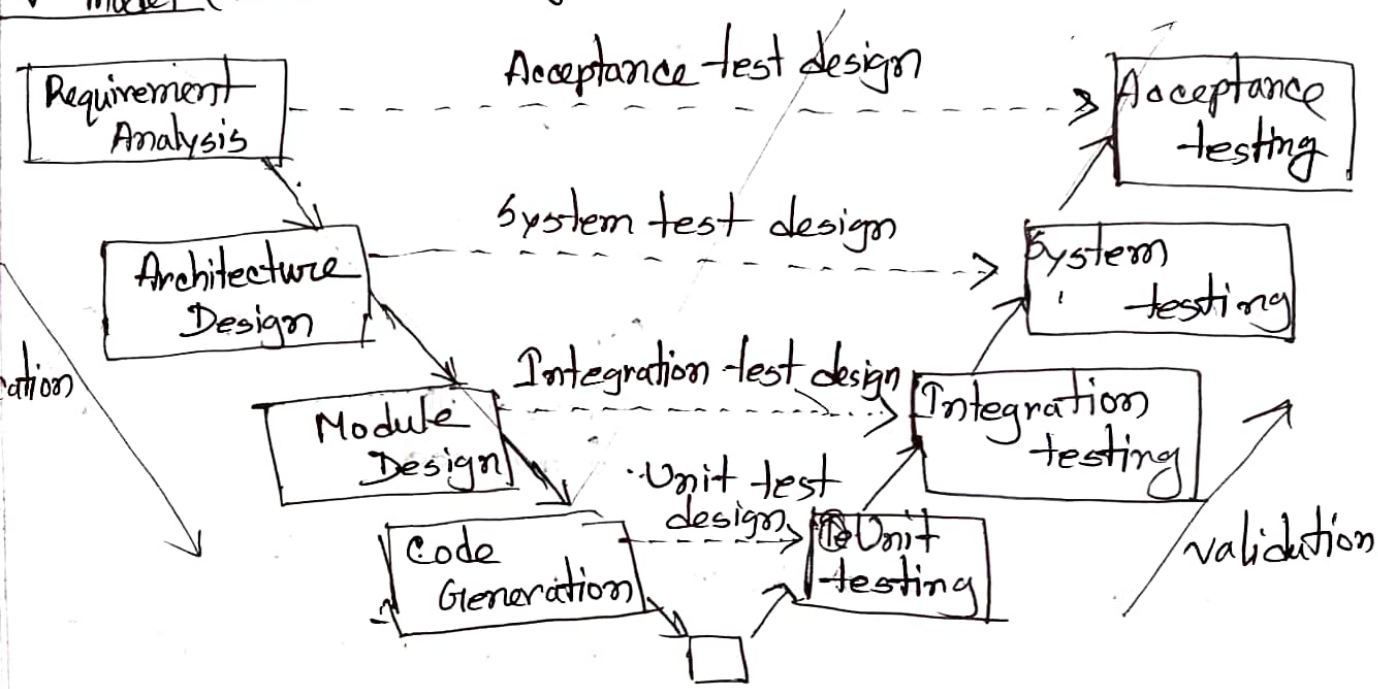
Disadvantage → 1. While completing a a stage, it freezes all the subsequent phase.

2. No way to varify the design.

3. Once testing is done, no features can be added.

4. Code reuse not possible.

V model (focuses on testing)



Advantage → 1. test activities planned before testing

2. Saves time

3. Emphasizes planning for V& V of the product.

Disadvantage → 1. Changes are not welcomed.

2. Software ~~dd~~ development is at the end.

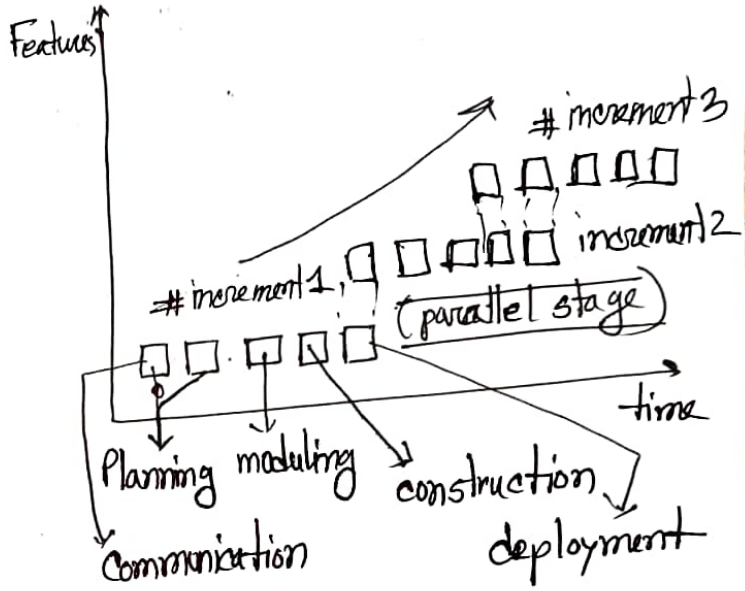
3. If any test fails then test document and code needs to be updated.

Incremental Process Model (Time, Quality of features ~~are~~ is priority)

A software process model where the software will be delivered in increments. (Fixed time + Fixed requirements)

Customer is well-known about the requirements.

- Customer wants to use the software from the begining.
- Software is devided into fixed number of increments to be delivered to customers.



When to use → 1. You are ready with thin slices of the ~~projec~~ software.

2. Customer wants prototype of the software from the beginning ~~+ quality is~~ & time is important.

3. Parallel stages such as collection, planning, modeling etc can take place.

4. Better for small, medium sized projects.

Disadvantages → 1. No edits/changes can be done after increments is done.

2. Code reuse not possible.

3. Can't trace back to any increments.

Iterative model (quality of the software is priority).

A software model where the software will be delivered in iteration.

~~→ Custor~~

When to use → 1. Customer is not sure about the requirements.

2. Development team is not sure about the technology, algorithm, tools to use.

3. Features are refined with time.

4. Customers feedback with repetitive iterations increased the product quality.

5. Better for long-term, complex projects.

Spiral Model (Rare)

It is a risk-driven project model generator for software projects. (High risk + unlimited budget.)

When to use → 1. Long term project commitment and budget.

2. No published material and experienced workers

3. Users and developers are unsure of the needs

4. New product line

5. Complex projects.

6. Risk are enormous.

In this model, we can see spiral loops showing step by step development, and risk analysis in every phase.

CMMI

~~The compatibility~~

The capability maturity model integration is a process and behavarial model that helps organization steamline software improvement and ~~inc~~ encourage productive and efficient behaviare that decrease risk in software, product and service development.

⇒ devided in 5 ~~stag~~ levels

Agile Methodology

Why we need it → For frequent requirement changes.
To provide smooth customer interactions.

Agile is a combination of iterative and incremental process models with focus on process adaptibility and customer satisfaction by ~~delivering~~ rapid deliver of working software.

Focuses on → individuals and interactions over process & tools
working software " comprehensive documentation
customer collaboration " contract nagotiation
responding to change " focusing on plan

Extreme programming → Also known as XP.
New version may be build every day.
Increments are delivered to the customer in every 2 weeks.
All test must run for build and ~~for~~ the build is accepted if ~~a all~~ the run is successfull.

Select user stories for releas → Break down story to task → Plan release → Develop rate/ test software → release software → Evaluate software → Select user stories for releas

Pair programming → Two people code in share, they switch roles from observer to navigator frequently.
- Better code and less cost.

Unit testing → Writes unit testing for each methods ~~for better code quality~~ to identify problems and resolve it.
TDD (Test first Development) is used to ensure better code quality and less error.
It refers to write code first and

after that write the development code.

Refactoring → It's a technique for restructuring existing code by changing it's internal structure without changing its external behaviour.

Restructuring includes reducing duplicate codes, breaking down long classes/methods in small ones, appropiate variable naming etc.

It improves design & extensibility of software.

Scrum (delivered in 14-30 days iterations) [most used]

product backlog → sprint planning meeting → sprint backlog → finished work

Product backlog → The requirement.

Priotized by product owner.

Repriotized at start of each sprint.

List of all desired work on project, ideally expressed as a list of users stories along with 'sprint points' (each item has a value)

Sprint backlog → High priority features from product backlog are selected based on points. These features will be delivered in 14 day. ~~Remaing~~ Any team member can add, change, delete sprint backlog.

Burndown chart → Display of completed and left to complete work.

Scrum Roles → Product owner: Product manager/project sponsor
Decides features, release date, prioritization, ~~mone~~ budget.

Scrum master: Project manager/team leader.
Enacts scrum values & practices

Project team: 5-10 members.
Project full development works.
Changes only between sprints

AUP (Agile unified process): Phases ⇒ Inception → Elaboration
↓
Transition ← Construction

Disciplines ⇒ Model → Implementation → test → Deployment
↓
Environment ← Configuration management.

CSE470

Functional requirements: (~~How~~ What the system should do.)

Statements of services the system should provide, how the system should react to particular inputs, and how the system should behave in particular situations.

⇒ Describes functionalities and system services.

⇒ Depends on the type of software, expected users and the type of system the software is used.

⇒ High level statements of ~~how the~~ what the should or shouldn't do.

⇒ Describes the system services in detail.

Non-functional requirements: (How the system should do)

Constraints on the services or functions offered by the system such as time constraints, development progree constraints etc.

⇒ Defines system properties and constraints such as, reliability, ~~&~~ response time and storage constraints etc.

⇒ Specifies mandating a particular IDE, programming language and development methods.

⇒ If not met, then system may be useless.

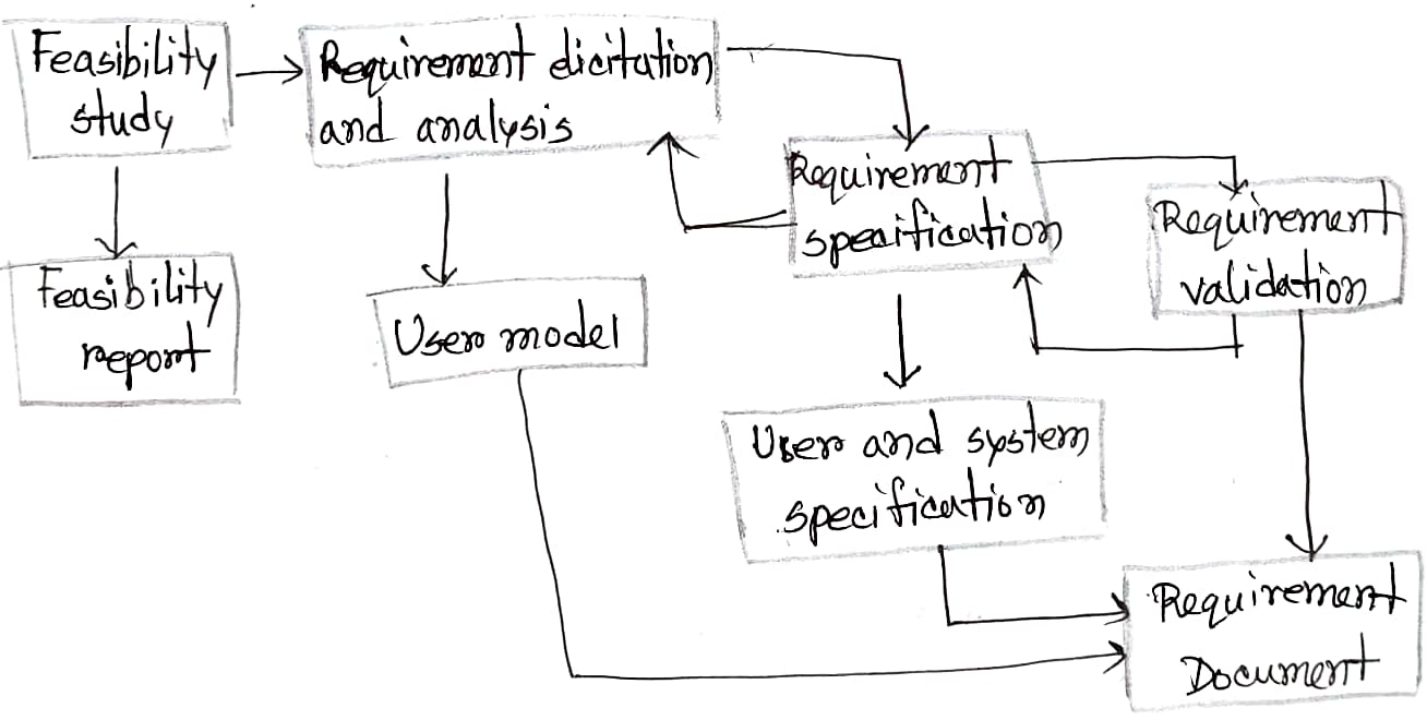
- Non-functional requirements
  - Product requirement (how the delivered product must behave)
    - Usability requirement
    - Reliability "
    - Dependability "
    - Security "
  - Organizational requirement (policies and procedures)
    - Environmental "
    - Operational "
    - Development "
  - External requirements
    - Regulatory "
    - Ethica "
    - Legislative "

	Functional	Non-function
1.	Defines the services or functions required by the ~~system~~ customer that must be provided by system.	Defines system properties or funct e.g. reliability, response time and storage requirments.
2.	Describes what the system should do.	Describes how the system will do.
3.	Related to business, ex: gross pay by the payroll department.	Related to improving the performance of the business.
4.	These are easy to test.	These are not easy to test.
5.	Related to individual system features.	Related to the system as a whole.
6.	Failure to meet these may degrade the system.	Failure to meet these may make the whole system unusab

Generic activities common to all requirements.

1. Rerequirements elicitation
2. " analysis
3. " validation
4. " management.

Requirement Engineering Process:



3.2.1

Software Architecture

Architectural view: There are 4-~~4~~ views from which architetture of software can be observed.

→ Logical view

→ Physical "

→ Development "

→ Process "

4+1 view model | → Logical view: Key abstractions in the system as objects/classes

Ex: Class/state diagram.

→ Process view: How at ~~a~~ run-time system is composed of interacting process

Ex: Activity diagram.

→ Development " : How the software is decomposed for development.

Ex: Component diagram.

→ Physical " : System hardware and how the software components are distributed in processor.

Ex: Deployment diagram.

MVC Pattern

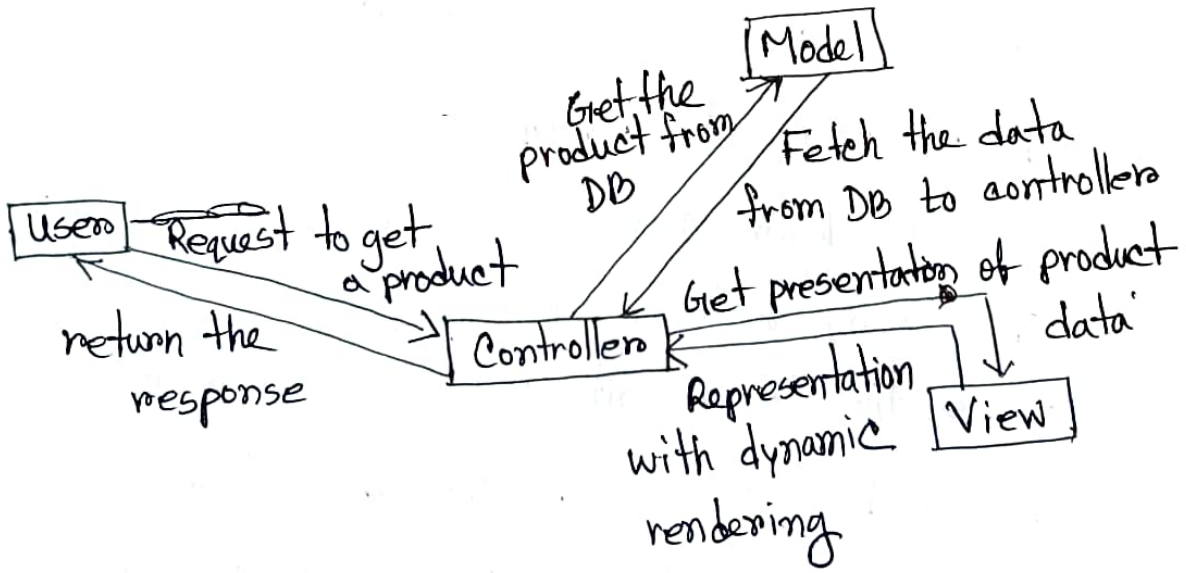
⇒ Model – View – Controller are three structured ~~pattern sep structed into three~~ logical components in MVC.

⇒ Separates presentation & interaction from data handling logic.

Model ⇒ 1. Refers to data related logic.
2. Interaction with database (Ex → UPDATE, DELETE, INSERT)
3. Communicates with controller.
4. Can update, collaborate with view.

View ⇒ 1. What end user see.
2. Consists of html, css
3. Communicates with controller.

Controller ⇒ 1. Handles all the user inputs through url.
2. Interaction with application starts with user interaction
3. Process http url requests → GET, POST, PUT, DELETE
4. Contains server side logic.
5. Communicates with View & Model.



Routes → URL's to access a resource.

When to use → 1. When there is multiple ways to view and interact with data.

2. When future requirements for interaction and presentation of data are unknown.

Advantages → 1. Allows data to change independently of it's presentation.

2. Supports presentation of same data in different ways with changes made.

Disadvantages → 1. Only for simple data model & interactions.

2. Can involve a code & code complexity when simple.

Layered Architecture

⇒ Organizes the system into set of layers, each of the layers provide a set of services.

⇒ Supports incremental development of sub-systems.

⇒ Change in layer interface, affects only the adjacent layer, as components are isolated;

⇒ Organizes the layers with related functionalities.

⇒ Layer provides services to the above layers, so the lowest layers represent core - services, that is to

be used throughout the system.

⇒ Layers are : Presentation layer
↑
Business "
↑
Persistence "
↑
Database "

→ these layers can be closed and can be opened ~~by~~ only by the above layer.

When to use → 1. When building new facilities on top of existing ones.
2. When development is spread accross several teams, each responsible for a layer.
3. When there is requirement for multi-level security.

Advantage → 1. Allows easy replacement/addition of layers.
2. Testing is easy.

Disadvantage → 1. Seperation of layers is difficult.
2. High-level layer may have to interact with low-level layers directly.
3. Performance problem.
4. A change in any layer requires to restart the application.

Repository Pattern

Mediates between domain and data mapping layers, like an in memory domain object collection.

⇒ Client objects construct query specifications and submits to repository for satisfiction. (Central repository)

⇒ Object can be ~~add~~ added & removed from repository.

⇒ The mapping code encapsulate by the repository will carry out operations behind the scenes.

⇒ Components do not interact directly but through repository.

When to used ⇒ 1. When large volume of information are generated that has to stored for a long time.

2. Data driven systems where inclusion of data in repository triggers an action/tool.

Advantage ⇒ 1. Problem in the repository will affect the whole system meaning (single point failure)

2. Inefficient

3. Distribution of repository across systems may be difficult.

Client-Server Pattern

The functionality of the system is organized into services.

⇒ Server provides services to multiple clients. components.

⇒ Client component requests services from server components.

When to use ⇒ 1. Data in shared database has to be accessed from different locations.

Advantages ⇒ 1. Servers can be distributed across a network.

2. General functionality can be available to all clients so doesn't need service implementation

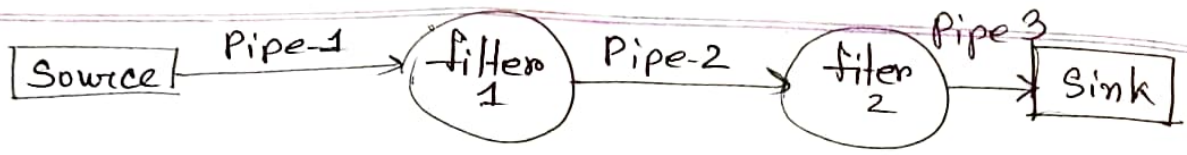
Disadvantages ⇒ 1. Each service is a single point of failure.

2. Performance is unpredictable since it depends on the network and system.

3. Management problems if servers are owned by different organizations.

Pipe-filter Pattern

The processing of the data in a system is organized so that each processing component (filter) is discrete & carries one type of data transformation. ~~Data fl~~



Here, stream of process and produced data. And, each processing ~~data~~ step is encapsulated in filter. Then, the processed data is passed through pipes. Pipes are used for buffering and synchronization.

When to use ⇒ 1. Data processing applications where inputs are processed in separate stages to generate output.

Advantages ⇒ 1. Easy to understand.
2. Supports transformation reuse.
3. Workflow style matches with business processes
4. Evolution by adding transformation is straightforward.
5. Implemented in sequential/concurrent system.

Disadvantages ⇒ 1. Formate for data transfer has to be agreed upon between communicating transform.
2. Each transformation must parse its input and unparse its output to the agreed form.
3. Increase system overhead.
~~4. Ir~~